

Guia de Instalação e Configuração via Docker: O-CU e O-DU (OCUDU)

Grupo de Pesquisa em Telecomunicações em Fio (GTEL)

13 de julho de 2026

Conteúdo

1	Introdução	2
2	Pré-requisitos	2
3	Cenário 1: Instalação Centralizada (All-in-One)	2
3.1	Preparação do Ambiente e Dependências	2
3.2	Estrutura do Docker Compose	2
3.3	Inicialização do Ambiente	4
3.4	Roteamento e Acesso à Internet para o UE	5
4	Cenário 2: Instalação Descentralizada	6
4.1	Topologia de Rede	6
4.2	Configuração do Host A (5G Core)	6
4.2.1	Configurando arquivo yml para open5gs	6
4.2.2	Alterando o arquivo open5gs.env	10
4.2.3	Alterando o arquivo open5gs_entrypoint.sh	11
4.2.4	Iniciando container open5gs	12
4.3	Configuração do Host B (gNB - OCUDU)	12
4.4	Configurando arquivo yml para gnb	13
4.5	Alterando arquivo .env (GRAFANA)	15
5	Monitoramento e Validação	15

1 Introdução

Este documento descreve o processo passo a passo de implantação dos componentes O-CU (Central Unit) e O-DU (Distributed Unit) utilizando virtualização baseada em containers. A abordagem via Docker garante isolamento, reprodutibilidade e agilidade no deploy da rede 5G Standalone. O guia aborda dois cenários: uma instalação centralizada (All-in-One) e uma descentralizada (Core e gNB distribuídos em hosts distintos).

2 Pré-requisitos

Antes de iniciar o deploy, certifique-se de que os hosts possuem o ambiente Docker devidamente configurado:

- Docker Engine (versão 20.10+ recomendada).
- Docker Compose V2 instalado.
- Entrada USB 3.0 ou superior (Necessária para conectar a USRP)

3 Cenário 1: Instalação Centralizada (All-in-One)

Neste cenário, todas as funções do 5G Core e da gNB (OCUDU) são instanciadas na mesma máquina hospedeira, de modo que os containers comuniquem-se internamente através de uma rede Docker dedicada em bridge.

3.1 Preparação do Ambiente e Dependências

O primeiro passo é clonar o repositório oficial do projeto e suas dependências, conforme detalhado no site oficial do OCUDU (https://docs.ocudu.org/user_manual/installation/). Em relação à biblioteca de processamento FFT, utilizaremos a `libfftw`, que é essencial para calcular as transformadas rápidas de Fourier usadas na modulação e demodulação OFDM. Além disso, como a gNB estará em execução nesta máquina e precisará acessar o hardware, é necessário instalar os drivers do rádio. Utilizaremos o UHD (USRP Hardware Driver), disponível no repositório da Ettus Research (<https://github.com/EttusResearch/uhd>).

3.2 Estrutura do Docker Compose

Após a preparação dos drivers e repositórios locais, navegue até o diretório `ocudu/docker/`. É neste local que se encontra o arquivo `docker-compose.yml` oficial, responsável por gerenciar o Core (Open5GS) e a gNodeB simultaneamente.

```
1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  #   Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
3
4 services:
5   5gc:
6     container_name: open5gs_5gc
7     build:
```

```

8     context: open5gs
9     target: open5gs
10    args:
11        OS_VERSION: "22.04"
12        OPEN5GS_VERSION: "v2.7.6"
13    env_file:
14        - ${OPEN_5GS_ENV_FILE:-open5gs/open5gs.env}
15    privileged: true
16    ports:
17        - "9999:9999/tcp"
18    # Uncomment port to use the 5gc from outside the docker
19    #     network
20    #     - "38412:38412/sctp"
21    #     - "2152:2152/udp"
22    command: 5gc -c open5gs-5gc.yml
23    volumes:
24        - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
25    healthcheck:
26        test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
27        interval: 3s
28        timeout: 1s
29        retries: 60
30    networks:
31        ran:
32            ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
33
34    gnb:
35        container_name: ocudu_gnb
36        # Build info
37        image: ocudu/gnb
38        build:
39            context: ..
40            dockerfile: docker/Dockerfile
41            args:
42                OS: ${OS:-ubuntu}
43                OS_VERSION: ${OS_VERSION:-26.04}
44        # privileged mode is required only for accessing usb devices
45        privileged: true
46        # Extra capabilities always required
47        cap_add:
48            - SYS_NICE
49            - CAP_SYS_PTRACE
50        volumes:
51            # Access USB to use some SDRs
52            - /dev/bus/usb/:/dev/bus/usb/
53            # Sharing images between the host and the pod.
54            - /usr/share/uhd/images:/usr/share/uhd/images
55            # Save logs and more into gnb-storage
56            - gnb-storage:/tmp
57        configs:
58            - gnb_config.yml

```

```

58     - gnb_compose_config.yml
59 networks:
60     ran:
61         ipv4_address: ${GNB_IP:-10.53.1.3}
62     metrics:
63         ipv4_address: 172.19.1.3
64     # Start GNB container after 5gc is up and running
65     depends_on:
66         5gc:
67             condition: service_healthy
68     # Command to run into the final container
69     command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml
70
71 configs:
72     gnb_config.yml:
73         file: ${GNB_CONFIG_PATH:-../configs/gnb_rf_b200_tdd_n78_20mhz
74             .yml}
75     gnb_compose_config.yml:
76         content: |
77             cu_cp:
78                 amf:
79                     addrs: ${OPEN5GS_IP:-10.53.1.2}
80                     bind_addrs: ${GNB_IP:-10.53.1.3}
81             metrics:
82                 autostart_stdout_metrics: true
83                 enable_json: true
84             remote_control:
85                 bind_addr: 0.0.0.0
86                 enabled: true
87
88 volumes:
89     gnb-storage:
90
91 networks:
92     ran:
93         ipam:
94             driver: default
95             config:
96                 - subnet: 10.53.1.0/24
97     metrics:
98         ipam:
99             driver: default
100            config:
101                - subnet: 172.19.1.0/24

```

Listing 1: docker-compose.yml oficial centralizado

3.3 Inicialização do Ambiente

Para subir a stack completa do Cenário 1, execute o comando abaixo no diretório `ocudu/docker/df`

```

1 # Inicializar os containers
2 docker compose up
3
4 # Verificar o status dos containers ativos
5 docker ps

```

Listing 2: Comandos para execução no Host Único

```

suporte@atlantico04067L:~/ocudu/docker$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
67a40e3a554c   docker-5gc    "./open5gs_entrypoin..." 8 minutes ago  Up 8 minutes (healthy)
0.0.0.0:9999->9999/tcp, [::]:9999->9999/tcp   open5gs_5gc
1e351f6cf198   ocudu/gnb     "gnb -c /gnb_config..." 15 minutes ago  Up 8 minutes
                                         ocudu_gnb

```

Figura 1: Containers em funcionamento

Para confirmar que a gNodeB estabeleceu comunicação com o Core 5G de forma bem-sucedida, é fundamental monitorar os logs do Core. Nesse sentido, o indicativo principal de sucesso na integração é a visualização da abertura da conexão SCTP, seguida imediatamente pela troca de mensagens de controle NGAP (*NG Application Protocol*). Especificamente, deve-se observar nos logs a chegada de uma mensagem **NG Setup Request** originada pela gNB e a respectiva resposta **NG Setup Response** emitida pelo Core. Essa validação comprova que a interface lógica N2 (plano de controle) está ativa, sincronizada e que a rede de acesso está pronta para iniciar o registro de equipamentos de usuário (UEs).

```

1 # Acompanhar os logs do 5G Core em tempo real
2 docker logs -f open5gs_5gc

```

Listing 3: Comando para monitoramento em tempo real do AMF no Core

```

07/08 23:03:19.719: [amf] INFO: gNB-N2 accepted[10.53.1.3]:52268 in ng-path module (./src/amf/ngap-sctp.c:113)
07/08 23:03:19.719: [amf] INFO: gNB-N2 accepted[10.53.1.3] in master_sm module (./src/amf/amf-sm.c:894)
07/08 23:03:19.733: [amf] INFO: [Added] Number of gNBs is now 1 (./src/amf/context.c:1277)
07/08 23:03:19.733: [amf] INFO: gNB-N2[10.53.1.3] max_num_of_ostreams : 30 (./src/amf/amf-sm.c:941)

```

Figura 2: Logs do Core (Open5GS)

3.4 Roteamento e Acesso à Internet para o UE

O passo final para que o Equipamento de Usuário (UE) obtenha conectividade externa (internet) através da rede 5G é configurar o roteamento e a tradução de endereços de rede (NAT). Esse procedimento é realizado dentro do container do 5G Core, que hospeda a função de plano de usuário (UPF) responsável por fazer a ponte entre a rede interna e a internet.

Primeiro, é necessário acessar o terminal interativo do container do Core. Embora seja possível listar os processos com `docker ps` e copiar o *Container ID*, a abordagem mais recomendada e automatizável é utilizar o nome exato do serviço:

```

1 # Acessar o container do Open5GS interativamente
2 sudo docker exec -it open5gs_5gc bash

```

Listing 4: Acesso ao terminal do container do 5G Core

Uma vez dentro do ambiente do container, os seguintes comandos devem ser executados para liberar o fluxo de pacotes na camada de rede:

```

1 # Habilitar o encaminhamento de pacotes no nível do kernel
2 sysctl -w net.ipv4.ip_forward=1
3
4 # Configurar a regra de NAT para mascarar o tráfego dos UEs
5 iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j
   MASQUERADE

```

Listing 5: Configuração de IP forwarding e NAT no container

4 Cenário 2: Instalação Descentralizada

Na arquitetura distribuída, o ambiente é dividido para simular um cenário real de Open-RAN. O 5G Core rodará no **Host A** e a gNB (OCUDU) rodará no **Host B**. A comunicação entre os containers agora trafega pela rede física subjacente.

4.1 Topologia de Rede

- **Host A (5G Core):** IP Físico 192.168.10.100
- **Host B (gNB - OCUDU):** IP Físico 192.168.10.200

4.2 Configuração do Host A (5G Core)

Renomeie o repositório para “ocudu_core”:

```

1 mv ocudu ocudu_core

```

Ademais, é necessário acessar a pasta do repositório instalado:

```

1 cd ocudu_core/docker

```

Liste os arquivos do repositório com `ls` e verifique os arquivos do repositório. Dentro da pasta `docker`, existe o arquivo “docker-compose.yml”:

```

1 sdr@epc-server:~/ocudu_core/docker$ ls
2 Dockerfile docker-compose.split.yml docker-compose.yml open5gs
   telegraf
3 README.md docker-compose.ui.yml grafana scripts

```

4.2.1 Configurando arquivo yml para open5gs

Abrindo o arquivo em questão:

```

1 nano docker-compose.yml

```

Teremos então a saída:

```
1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
3 services:
4   5gc:
5     container_name: open5gs_5gc
6     build:
7       context: open5gs
8       target: open5gs
9       args:
10        OS_VERSION: "22.04"
11        OPEN5GS_VERSION: "v2.7.6"
12     env_file:
13       - ${OPEN_5GS_ENV_FILE:-open5gs/open5gs.env}
14     privileged: true
15     ports:
16       - "9999:9999/tcp"
17     # Uncomment port to use the 5gc from outside the docker
18     network
19     # - "38412:38412/sctp"
20     # - "2152:2152/udp"
21     command: 5gc -c open5gs-5gc.yml
22     volumes:
23       - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
24     healthcheck:
25       test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
26       interval: 3s
27       timeout: 1s
28       retries: 60
29     networks:
30       ran:
31         ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
32     ..[codigo continua]..
```

Se atendo ao serviço 5gc, para que o container fique exposto à rede do host, é necessário acrescentar o modo de rede igual ao do host:

```
1 network_mode: host
```

Alterando:

```
1 services:
2   5gc:
3     container_name: open5gs_5gc
4     network_mode: host
5     build:
6     ..[codigo continua]..
```

Incluindo variável HOSTNAME:

```
1     privileged: true
2     environment:
```

```

3      - HOSTNAME=0.0.0.0
4      ports:...[codigo continua]..
5  gnb:
6      container_name: ocudu_gnb
7      # Build info
8      image: ocudu/gnb
9      build:
10     ..[codigo continua]..

```

Por último, a seção **networks** deve ser comentada:

```

1      healthcheck:
2          test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
3          interval: 3s
4          timeout: 1s
5          retries: 60
6      # networks:
7      #   ran:
8      #     ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
9      ..[codigo continua]..

```

No fim do arquivo existe a declaração de criação de redes **network** e **metrics**, que devem ser comentadas visto o container estar em modo host:

```

1  #networks:
2  #   ran:
3  #     ipam:
4  #       driver: default
5  #       config:
6  #         - subnet: 10.53.1.0/24
7  #   metrics:
8  #     ipam:
9  #       driver: default
10  ##       config:
11  #         - subnet: 172.19.1.0/24
12  ..[codigo continua]..

```

Se deve comentar também toda a seção **gnb**, pois este host será apenas para o **CORE**, devendo estar dessa forma com as modificações:

```

1  services:
2      5gc:
3          container_name: open5gs_5gc
4          network_mode: host
5          build:
6              context: open5gs
7              target: open5gs
8              args:
9                  OS_VERSION: "22.04"
10                 OPEN5GS_VERSION: "v2.7.6"
11          env_file:
12              - ${OPEN_5GS_ENV_FILE:-open5gs/open5gs.env}
13          privileged: true
14          environment:

```



```

15     - HOSTNAME=0.0.0.0
16   ports:
17     - "9999:9999/tcp"
18   # Uncomment port to use the 5gc from outside the docker
19   # - "38412:38412/sctp"
20   # - "2152:2152/udp"
21   command: 5gc -c open5gs-5gc.yml
22   volumes:
23     - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
24   healthcheck:
25     test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
26     interval: 3s
27     timeout: 1s
28     retries: 60
29   # networks:
30   #   ran:
31   #     ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
32   ..[codigo continua]..

```

```

#gnb:
#container_name: ocudu_gnb
# Build info
#image: ocudu/gnb
#build:
#context: ..
#dockerfile: docker/Dockerfile
#args:
#OS: ${OS:-ubuntu}
#OS_VERSION: ${OS_VERSION:-26.04}
# privileged mode is required only for accessing usb devices
#privileged: true
# Extra capabilities always required
#cap_add:
#- SYS_NICE
#- CAP_SYS_PTRACE
#volumes:
# Access USB to use some SDRs
#- /dev/bus/usb/:/dev/bus/usb/
# Sharing images between the host and the pod.
# It's also possible to download the images inside the pod
#- /usr/share/uhd/images:/usr/share/uhd/images
# Save logs and more into gnb-storage
#- gnb-storage:/tmp
# It creates a file/folder into /config_name inside the container
# Its content would be the value of the file used to create the config
#configs:
#- gnb_config.yml
#- gnb_compose_config.yml
# Customize your desired network mode.
# current network configuration creates a private network with both containers attached
# An alternative would be `network: host`. That would expose your host network into the container
#networks:
#ran:
#ipv4_address: ${GNB_IP:-10.53.1.3}
#metrics:
#ipv4_address: 172.19.1.3
# Start GNB container after 5gc is up and running

```

Figura 3: Começo do campo gnb

```

# An alternative would be `network: host`. That would expose your host network into the container
#networks:
#  ran:
#    ipv4_address: ${GNB_IP:-10.53.1.3}
#metrics:
#  ipv4_address: 172.19.1.3
# Start GNB container after 5gc is up and running
#depends_on:
#  5gc:
#    condition: service_healthy
# Command to run into the final container
#command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml

#configs:
#gnb_config.yml:
#  file: ${GNB_CONFIG_PATH:-../configs/gnb_rf_b200_tdd_n78_20mhz.yml} # Path to your desired config
#gnb_compose_config.yml:
#  content: |
#    cu_cp:
#      amf:
#        addrs: ${OPEN5GS_IP:-10.53.1.2}
#        bind_addrs: ${GNB_IP:-10.53.1.3}
#      metrics:
#        autostart_stdout_metrics: true
#        enable_json: true
#      remote_control:
#        bind_addr: 0.0.0.0
#        enabled: true

#volumes:
#gnb-storage:

```

Figura 4: Final do campo gnb

4.2.2 Alterando o arquivo open5gs.env

Além de modificar o arquivo docker-compose.yml, é necessário mudar outro arquivo chamado open5gs.env.

Para acessá-lo:

```

1 cd ocudu_core/docker/open5gs
2 nano open5gs.env

```

A saída do comando será:

```

1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
3 MONGODB_IP=127.0.0.1
4 OPEN5GS_IP=10.53.1.2
5 UE_IP_BASE=10.45.0
6 UPF_ADVERTISE_IP=10.53.1.2
7 DEBUG=false
8 SUBSCRIBER_DB=001010123456780,00112233445566778899aabbccddeeff,
  opc,63bfa50ee6523365ff14c1f45f88737d,8000,9,10.45.1.2
9 NETWORK_NAME_FULL=OCUDU
10 NETWORK_NAME_SHORT=OCUDU
11 # Timezone - this setting will also be conveyed by Open5GS
  towards UE via "configuration update command"; comment for UTC
12 TZ=Europe/Madrid

```

Será necessário mudar os espaços OPEN5GS_IP e UPF_ADVERTISE_IP:

```

1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI

```

```

3 MONGODB_IP=127.0.0.1
4 OPEN5GS_IP=192.168.152.131
5 UE_IP_BASE=10.45.0
6 UPF_ADVERTISE_IP=192.168.152.131
7 DEBUG=false
8 SUBSCRIBER_DB=0010100000000008,465B5CE8B199B49FAA5FOA2EE238A6BD,
   opc,E8ED289DEBA952E4283B54E88E6183CB,8000,9,10.45.1.2
9 NETWORK_NAME_FULL=OCUDU
10 NETWORK_NAME_SHORT=OCUDU
11 # Timezone - this setting will also be conveyed by Open5GS
   towards UE via "configuration update command"; comment for UTC
12 TZ=America/Fortaleza

```

No caso, como o modo de rede está configurado para host, OPEN5GS_IP é o mesmo IP do host.

4.2.3 Alterando o arquivo open5gs_entrpoint.sh

Para acessar o arquivo:

```

1 cd ocudu_core/docker/open5gs
2 nano open5gs_entrpoint.sh

```

Na saída:

```

1 #! /bin/bash
2 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
   Systems Limited
3 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
4 export UE_GATEWAY_IP="${UE_IP_BASE}.1"
5 export UE_IP_RANGE="${UE_IP_BASE}.0/24"
6 INSTALL_ARCH=x86_64-linux-gnu
7 if [ "$(uname -m)" = "aarch64" ]; then
8     INSTALL_ARCH="aarch64-linux-gnu"
9 fi
10 export INSTALL_ARCH
11 envsubst < open5gs-5gc.yml.in > open5gs-5gc.yml
12 # create dummy interfaces on localhost ip range for open5gs
   entities to bind to
13 for IP in {2..22}
14 do
15     ip link add name lo$IP type dummy
16     ip ad ad 127.0.0.$IP/24 dev lo$IP
17     ip link set lo$IP up
18 done
19 ..[codigo continua]..

```

No bloco onde as interfaces de rede dummy são criadas, é necessário mudar a máscara CIDR de 24 para 32. Ademais, deve ser feita a adição de rotas e habilitar o encaminhamento, dessa forma os UEs irão conseguir ter acesso à internet.

```

1 #! /bin/bash
2 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
   Systems Limited

```

```

3 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
4 export UE_GATEWAY_IP="${UE_IP_BASE}.1"
5 export UE_IP_RANGE="${UE_IP_BASE}.0/24"
6 INSTALL_ARCH=x86_64-linux-gnu
7 if [ "$(uname -m)" = "aarch64" ]; then
8     INSTALL_ARCH="aarch64-linux-gnu"
9 fi
10 export INSTALL_ARCH
11 sysctl -w net.ipv4.ip_forward=1
12 sysctl -w net.ipv6.conf.all.forwarding=1
13 iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j
    MASQUERADE
14 ip6tables -t nat -A POSTROUTING -s 2001:db8:cafe::/48 ! -o ogstun
    -j MASQUERADE
15 iptables -A FORWARD -i ogstun -j ACCEPT
16 iptables -A FORWARD -o ogstun -m state --state RELATED,
    ESTABLISHED -j ACCEPT
17 envsubst < open5gs-5gc.yml.in > open5gs-5gc.yml
18 # create dummy interfaces on localhost ip range for open5gs
    entities to bind to
19 for IP in {2..22}
20 do
21     ip link add name lo$IP type dummy
22     ip ad #ad 127.0.0.$IP/32 dev lo$IP
23     ip link set lo$IP up
24 done
25 ..[codigo continua]..

```

4.2.4 Iniciando container open5gs

Para inicializar o container do open5gs, basta estar na pasta docker e aplicar o comando:

```

1 cd ocudu_core/docker
2 sudo docker compose up 5gc

```

Para verificar se o container está online basta utilizar `sudo docker ps`.

```

sdr@epc-server:~/ocudu/docker$ sudo docker ps
[sudo] password for sdr:

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
a68c86ceae0	docker-5gc	"./open5gs_entrypoint..."	31 minutes ago	Up 31 minutes (healthy)	
open5gs_5gc					

Figura 5: docker ps no Core

4.3 Configuração do Host B (gNB - OCUDU)

Nessa etapa do tutorial, o foco é mudado para o host que hospedará o container da gnb. Dessa forma, primeiramente o nome do repositório ocudu baixado deve ser mudado para `ocudu_gnb`.

```

1 mv ocudu ocudu_gnb

```

Agora, para a próxima etapa, acesse a pasta docker:

```
1 cd ocudu_gnb/docker
```

4.4 Configurando arquivo yml para gnb

Acessando o arquivo docker-compose.yml:

```
1 nano docker-compose.yml
```

É necessário que o container da gnb, assim como o container do open5gs, funcionem em modo de rede host. Dessa forma, é preciso acrescentar ao arquivo a configuração `network_mode: host`.

```
1 ..[codigo continua]..
2 gnb:
3   container_name: ocudu_gnb
4   network_mode: host
5   # Build info
6   image: ocudu/gnb
7   build:
8   ..[codigo continua]..
```

Além disso, os endereços IPs (`amf: addrs`, `bind_addrs`) presentes no arquivo devem ser mudados de acordo com o endereço do host de cada container. Assim, como feito no arquivo yml para o 5gc, as declarações de rede devem ser comentadas.

```
1 networks:
2   ran:
3     ipv4_address: ${GNB_IP:-10.53.1.3}
4     metrics:
5       ipv4_address: 172.19.1.3
6   # Start GNB container after 5gc is up and running
7   depends_on:
8     5gc:
9       condition: service_healthy
10  # Command to run into the final container
11  command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml
12  configs:
13    gnb_config.yml:
14      file: ${GNB_CONFIG_PATH:-../configs/
15          gnb_rf_b200_tdd_n78_20mhz.yml}
16    gnb_compose_config.yml:
17      content: |
18        cu_cp:
19          amf:
20            addrs: ${OPEN5GS_IP:-10.53.1.2}
21            addrs: ${OPEN5GS_IP:-192.168.152.131}
22            bind_addrs: ${GNB_IP:-10.53.1.3}
23            bind_addrs: ${GNB_IP:-192.168.152.140}
24          metrics:
25            autostart_stdout_metrics: true
26            enable_json: true
```

```

26         remote_control:
27             bind_addr: 0.0.0.0
28             enabled: true
29     volumes:
30         gnb-storage:
31     #networks:
32     #     ran:
33     #         ipam:
34     #             driver: default
35     #             config:
36     #                 - subnet: 10.53.1.0/24
37     #     metrics:
38     #         ipam:
39     #             driver: default
40     #             config:
41     #                 - subnet: 172.19.1.0/24
42 ..[codigo continua]..

```

Assim como no **CORE**, se deve comentar todo o campo **5GC**, pois esse host será apenas **gnb**. Lembrando que com essas alterações se deve manter o comando *command* descomentado. Ficando da seguinte forma:

```

services:
#COMENTADO NO DOCUMENTO
# 5gc:
#     container_name: open5gs_5gc
#     build:
#         context: open5gs
#         target: open5gs
#         args:
#             OS_VERSION: "22.04"
#             OPEN5GS_VERSION: "v2.7.6"
#     env_file:
#         - ${OPEN5GS_ENV_FILE:-open5gs/open5gs.env}
#     privileged: true
#     ports:
#         - "9999:9999/tcp"
#     # Uncomment port to use the 5gc from outside the docker network
#     # - "38412:38412/sctp"
#     # - "2152:2152/udp"
#     command: 5gc -c open5gs-5gc.yml
#     volumes:
#         - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
#     healthcheck:
#         test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
#         interval: 3s
#         timeout: 1s
#         retries: 60
#     networks:
#         ran:
#             ipv4_address: ${OPEN5GS_IP:-10.53.1.2}

```

Figura 6: Começo do campo 5gc


```

#   networks:
#   ran:
#       ipv4_address: ${GNB_IP:-10.53.1.3}
#   metrics:
#       ipv4_address: 172.19.1.3
#   # Start GNB container after 5gc is up and running
#   depends_on:
#       5gc:
#           condition: service_healthy
#   # Command to run into the final container
#   command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml

```

Figura 7: Final do campo 5gc)

Para verificar se o container está online basta utilizar `sudo docker ps`.

```

sdr@gnb-server:~/ocudu/docker$ sudo docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
61dbcb985154	ocudu/gnb	"gnb -c /gnb_config..."	5 minutes ago	Up 5 minutes		ocudu_gnb

Figura 8: docker ps no gnb

4.5 Alterando arquivo .env (GRAFANA)

Para acessar o arquivo, basta seguir os caminhos abaixo para o host com o container da gnb:

```

1 cd ocudu_gnb/docker
2 nano .env

```

OBS: utilizando somente o comando `ls` para listar os arquivos na pasta docker não é possível visualizar o arquivo `.env`. Para isso basta utilizar o comando `ls -a`.

Como o container gnb funcionará em outro host, aqui deve ser colocado o IP do host que contém gnb. Dessa forma:

```

1 # Default: gNB running inside Docker (Option A)
2 WS_URL=gnb:8001
3 WS_URL=192.168.152.140:8001
4 ..[codigo continua]..

```

5 Monitoramento e Validação

Para validar se a associação NGAP entre o container do OCUDU e o container do Core foi estabelecida com sucesso, utilize as ferramentas de inspeção de logs e redes do Docker.

```

1 # Verificar os logs em tempo real da gNB no Host B
2 docker logs -f ocudu_gnb
3
4 # Inspeccionar estatísticas de rede do container do Core no Host A
5 docker stats open5gs_5gc

```

Listing 6: Comandos úteis de troubleshooting